

TradeFlow

An EXIOBASE Pipeline Field Guide

Trade. Factor. Industry.

How multi-regional input-output data becomes queryable trade and environmental impact.

Model.Earth · [exiobase/tradeflow](https://exiobase.org/tradeflow)

This volume is an auto-generated guide to the **exiobase/tradeflow** Python pipeline inside the Model.Earth repository. It covers the data model, the processing scripts, the configuration surface, and the US BEA and India extensions. It is a *field guide*, not a formal specification — cross-reference the source files when the two disagree.

All matrix mathematics follows the standard EXIOBASE v3 convention: **Z** is inter-industry transaction flows (M EUR), **Y** is final demand, and **F** / **S** hold environmental extensions (absolute and per-M-EUR intensities respectively).

Contents

Part I — Foundations

- 1 What TradeFlow is, and why it exists
- 2 Three matrices, three CSVs, and a diagram
- 3 The vocabulary: trade, factor, industry, level

Part II — The Core Pipeline

- 4 Configuration: config.yaml and config_loader.py
- 5 trade.py: Extraction and factor generation
- 6 trade_impact.py: Aggregation
- 7 trade_resource.py: Three specialised views
- 8 main.py: Batch orchestration

Part III — Reference and Companion Files

- 9 factor.csv and industry.csv
- 10 The 120-vs-721 selection problem
- 11 CSV output catalogue

Part IV — Extensions

- 12 US BEA: Interstate state-to-state flows
- 13 India: State-level allocation
- 14 SQL ingestion (insert_sql.py)

Part V — Operations

- 15 Running it: commands, timing, resumability
- 16 Troubleshooting and gotchas
- 17 The schema being proposed for SQL
- Appendix A — Repository map
- Appendix B — Glossary

CHAPTER 1

What TradeFlow is, and why it exists

TradeFlow is the Python pipeline that lives at `exiobase/tradeflow/` in the Model.Earth repository. Its job is to turn the raw EXIOBASE v3 multi-regional input-output (MRIO) database into a set of flat, relational CSVs that downstream tools — SQL databases, dashboards, Sankey diagrams, interstate trade maps — can consume without ever touching the original matrices.

The problem

EXIOBASE ships as a zip of about 535 MB per year. Inside are large, sparse matrices indexed by (region, sector) tuples: around 44 regions × 200 products on each axis. A single matrix cell is a monetary flow or an environmental coefficient. These are wonderful for linear algebra and terrible for everything else. You cannot easily answer a question like ‘what are the top ten CO₂ emission sources embedded in US imports from China in 2019?’ by inspection.

The approach

TradeFlow **unstacks** the matrices into long tables with one row per *trade transaction* (a flow from one region/industry to another) and one row per *trade-factor relationship* (how much of a given environmental stressor is embedded in that transaction). Each row carries a stable identifier so the tables can be joined again downstream. The output is a tree of CSV files organised by year / country / flow_direction.

Who it is for

Three audiences, in order of priority:

- **Data engineers** who want to load trade and impact data into Postgres or SQL Server without re-parsing the pymrio pickle every time.
- **Policy analysts and researchers** who want to filter trade flows by region, industry, or environmental factor and plot the results.

- **Downstream applications** — Sankey diagrams, interstate trade maps, LCA dashboards — that fetch pre-computed CSVs from GitHub rather than running the pipeline themselves.

What makes TradeFlow worth documenting

Three design decisions are not obvious from reading the code and are the main subject of this book. First, the pipeline is **config-driven**: a single `config.yaml` file controls year, country, flow direction, factor-selection strategy, and output paths. Second, it ships a **dual-file** output strategy for environmental factors (120 selected vs 721 full) that tries to balance coverage against Node.js and SQL memory limits. Third, it is **resumable**: the batch runner skips countries whose `runnote.md` says they are already done, and timeout tiers (per-script, per-country, per-batch) stop runaway work.

CHAPTER 2

Three matrices, three CSVs, and a diagram

Before diving into any single file, it helps to carry a mental model of what the pipeline is actually doing. EXIOBASE provides three matrices. TradeFlow consumes them and emits three families of CSVs. The rest is bookkeeping.

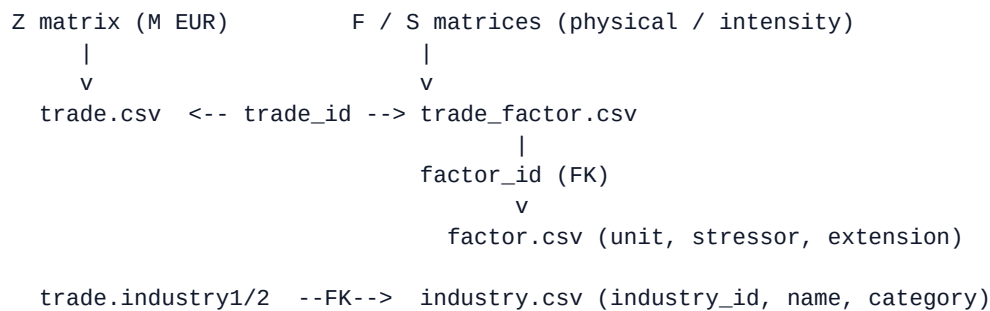
The three matrices

Matrix	What it holds
Z	Inter-industry monetary flows. $Z[i,j]$ is the value in M EUR flowing from (region, sector) i to (region, sector) j .
Y	Final demand. $Y[i,c]$ is the value flowing from industry i into consumption category c (households, government, investment).
F / S	Environmental extensions. F is absolute stressor emissions by (region, sector); S is the per-M-EUR intensity. There are six extensions: air_emissions, employment, energy, land, material, water.

The three output families

CSV family	Purpose
trade.csv	One row per bilateral industry-pair flow for a given country and direction. Columns: trade_id, year, region1, region2, industry1, industry2, amount (M EUR).
trade_factor.csv trade_factor_lg.csv	/ One row per (trade, factor) pair. Columns: trade_id, factor_id, level. The level is a physical quantity (kg, persons, TJ, km ² , kt, Mm ³).
trade_impact / resource / material / employment	/ Pre-aggregated analytical views. These can be rebuilt from trade × trade_factor × factor, so when the data is in SQL they become VIEWS rather than tables.

The diagram



Everything else in the pipeline is a transformation, aggregation, or extension of this core shape. Interstate (US BEA) adds a parallel pair of tables that follow the same pattern but at the state level. India adds a parallel pair at the state/UT level. SQL ingestion loads these shapes directly.

CHAPTER 3

The vocabulary: trade, factor, industry, level

The TradeFlow authors deliberately chose plain-English table names — *‘designed for third graders’* is a direct quote from the README. This chapter unpacks the vocabulary because the terminology diverges from both academic MRIO literature and from EXIOBASE's native terms.

Trade (not ‘flow’)

A **trade** row is a single monetary flow from an origin (region1, industry1) to a destination (region2, industry2) within a single year. It is sourced directly from the EXIOBASE Z matrix. In the pymrio and MRIO communities this is universally called a *flow*; TradeFlow renames it to **trade** because ‘flow’ is overloaded (the word is also used for environmental flows in FEDEFL, and for Sankey diagram arcs). **trade.amount** is always in **million euros** (M EUR).

Factor

A **factor** is a single environmental stressor — one row in the EXIOBASE extension tables. There are **721 factors** total across six extensions. Each factor has a stable integer **factor_id** assigned by row position when `factors.py` walks the extensions in a fixed order: `air_emissions`, `employment`, `energy`, `land`, `material`, `water`. Factors carry a **unit** and belong to an **extension**.

Industry

An **industry** is a product-by-product EXIOBASE sector — there are around 200 of them. TradeFlow assigns each a 5-character **industry_id** derived from the sector name by `create_sector_mapping.py`. These codes are intended to be mnemonic (e.g. WHEAT for ‘Wheat’) but collisions are resolved by overwriting the last character with a counter.

Level (vs amount, vs coefficient)

This is the trickiest distinction in the whole system. There are three numeric columns across the pipeline, and getting them confused is the single most common source of bugs.

Column	What it is
trade.amount	Monetary value of a trade in M EUR. Comes from Z matrix.
coefficient	Physical units per M EUR of output. Comes from S matrix. Not stored in output CSVs — it is derivable as <code>level / amount</code> .
trade_factor.level	Physical impact of a trade on a factor. $\text{level} = \text{amount} \times \text{coefficient}$. Unit depends on the factor's extension.

The authors chose **level** (rather than `flow_value`, `impact_value`, or `levelX`) to signal that this is a physical quantity, not money. The same convention is used in `interstate_factor.level` for state-to-state flows.

The unit table

Each of the six EXIOBASE extensions has its own canonical unit. The unit for any given **level** is read from `factor.csv` by joining on `factor_id`.

Extension	Unit
air_emissions	kg
employment	1000 persons (people) / M.hr (hours)
energy	TJ (terajoules)
land	km ²
material	kt (kilotonnes)
water	Mm ³ (million cubic metres)

CHAPTER 4

Configuration: config.yaml and config_loader.py

Every single script in the pipeline reads from the same `config.yaml`. There are no command-line flags for year, country, or flow direction — those live in the config. The design intent is that *'edit the YAML and run python main.py'* is the entire user interface.

The config.yaml file

```
YEAR: 2019
TRADEFLOW: domestic,imports,exports
COUNTRY:
  list: US

FOLDERS:
  base: ../../trade-data/year/{year}
  imports: ../../trade-data/year/{year}/{country}/imports
  exports: ../../trade-data/year/{year}/{country}/exports
  domestic: ../../trade-data/year/{year}/{country}/domestic

FILES:
  factors: factor.csv
  industries: industry.csv
  industryflow: trade.csv
  trade_factor: trade_factor.csv
  trade_factor_domestic: trade_factor_lg.csv
  trade_impact: trade_impact.csv
  trade_resource: trade_resource.csv
  ...

PROCESSING:
  min_impact_threshold: 0.001
  use_partial_factors: true
  partial_factor_limit: 120
  use_partial_factors_domestic: false
  partial_factor_limit_domestic: 2000
  use_partial_factors_interstate: true
  partial_factor_limit_interstate: 5
```

COUNTRY: list vs current

The COUNTRY field has three valid shapes:

- **Explicit list:** `list: CN, DE, JP` — a comma-separated list.
- **Default set:** `list: default` — the 12 built-in countries (AU, BR, CA, CN, DE, FR, GB, IN, IT, JP, KR, US).
- **All:** `list: all` — auto-discovers existing country folders under `year/{year}/`.

When `main.py` runs, it writes a `current: sub-field` into the config so that subprocess scripts know which single country to work on. This mutation is **not safe for concurrent runs** — two batch runners on the same config will clobber each other's `current`.

Environment variable overrides

`config_loader.py` accepts `EXIOBASE_TRADEFLOW` and `EXIOBASE_COUNTRY` environment variables that override the YAML values. `main.py` sets these for each subprocess invocation, which neutralises the race condition above for the batch runner's own subprocesses.

Smart file selection

`get_file_path(config, 'trade_factor')` is more clever than it looks. For the domestic flow direction it first checks whether a `_lg.csv` file exists and prefers that over the standard version; for imports and exports it always uses the small file. This is where the *'120 for international, 721 for domestic'* policy is enforced.

CHAPTER 5

trade.py: Extraction and factor generation

`trade.py` is the heart of the pipeline. It is also, at ~1200 lines, the largest and most intricate script. Its responsibilities are:

- Download the EXIOBASE zip from Zenodo if it is not already in `exiobase_data/`.
- Create `industry.csv` if missing (delegates to `create_sector_mapping.py`).
- Create `factor.csv` if missing (delegates to `factors.py`).
- Parse the Z matrix, stack it long, filter by flow direction, and write `trade.csv`.
- Parse the S matrix, apply the 120-factor selection (unless `-lag` is passed), merge against `trade`, compute `level = amount × coefficient`, and write `trade_factor.csv`.

The ExiobaseTradeFlow class

The script wraps everything in a class whose constructor downloads the data, loads the sector mapping, and ensures `factor.csv` exists. A single instance handles one (year, country, flow-direction) tuple at a time — the batch runner re-instantiates the class per country.

Three flow directions

Direction	Filter rule
imports	<code>region2 == country AND region1 != country</code> . Keeps rows with <code>amount > 0.01 M EUR</code> .
exports	<code>region1 == country AND region2 != country</code> . Keeps rows with <code>amount > 0.01 M EUR</code> .
domestic	<code>region1 == country AND region2 == country</code> . Keeps rows with <code>amount > 0.001 M EUR</code> .

The thresholds are hard-coded. The domestic threshold is ten times finer because intra-country flows are typically small individual rows that still matter in aggregate.

The 120-factor selection

When `use_partial_factors: true` (the default for non-domestic flows), `_apply_partial_factors_filter` performs a two-stage selection per extension:

- **Priority first:** a hard-coded list per extension (e.g. CO₂, CH₄, N₂O, NO_x for air_emissions; Employment people, Employment hours for employment) is matched by substring.
- **Magnitude next:** if there are more than `partial_factor_limit` priority matches, keep the top-N by `|coefficient|`. If fewer, pad with other factors ranked by `|coefficient|`.

The result is written to `trade_factor.csv`. With `-lag`, the filter is skipped and the full 721-factor join is written to `trade_factor_lg.csv` instead (~1.5 GB, with a warning about downstream Node.js memory errors).

The merge, chunked

The join of trade rows against factor coefficients can easily exceed memory, so `trade.py` processes trades in **10,000-row chunks**, merging each chunk against the pre-indexed `F_stacked` frame. For a US exports run with ~189,000 trade rows and ~125,000 factor rows per extension, this takes about two minutes end-to-end.

level unit handling

After the merge, `level` is rounded by extension:

- **water, air_emissions:** 3 decimal places (small physical quantities).
- **all others:** rounded to integer.
- Rows with $|level| \leq 0.001$ are dropped as noise.

This rounding is *destructive* — the coefficient cannot be perfectly recovered from `level/amount` after rounding. The BEA documentation acknowledges this: *‘back-deriving coefficient is approximate’*.

CHAPTER 6

trade_impact.py: Aggregation

`trade_impact.py` is the second script in the `main.py` pipeline. Where `trade.py` produces one row per (trade, factor) pair, `trade_impact.py` collapses that back down to one row per trade transaction, with environmental impacts spread across columns.

What it reads

- `trade.csv` — the trade flows from `trade.py`.
- `trade_factor.csv` or `trade_factor_lg.csv` — whichever one `get_file_path` selects for this flow direction.
- `factor.csv` — for the unit, extension, and stressor columns.

What it computes

The script groups by `trade_id` and emits a wide table with roughly 22 columns:

- **Summary stats:** `total_level`, `factor_count`, `unique_factors`.
- **Extension pivot:** one column per extension (air_emissions, water, land, material, energy, employment) holding the sum of levels in that extension.
- **Major factor types:** `CO2_total`, `CH4_total`, `N2O_total`, `NOX_total`, `Water_total`, `Energy_total`, `Land_total` — computed by substring-matching stressor names.
- **Employment, two columns:** `Employment_people_total` (converted from '1000 p' to actual people by multiplying by 1000) and `Employment_hours_total` (left in millions of hours).
- **impact_intensity** = `total_level / amount`, with `inf/-inf` replaced by 0.

The employment unit bug (fixed)

EMPLOYMENT_FIX_SUMMARY.md documents a historical bug where `trade_impact.py` summed 'Employment people' (1000 p) and 'Employment hours' (M.hr) factors into a single `Employment_total` column. This produced nonsense — literally more than 600 billion employed people for China. The fix splits them into two columns and converts people to units of 1 (not 1000). If you see legacy outputs with an `Employment_total` column, they are miscalibrated.

Why it is a candidate for a VIEW

Every column in `trade_impact.csv` is a deterministic aggregation of `trade × trade_factor × factor`. The `PLAN.md` roadmap recognises this and marks the file as *'skipped — computable via JOIN'* for SQL ingestion. In other words, `trade_impact.csv` exists for static dashboards and GitHub-hosted CSV consumers; it does not need to live in the database.

CHAPTER 7

trade_resource.py: Three specialised views

trade_resource.py is the third main.py script. It splits trade_factor rows into three domain-specific files — employment, resources, and materials — each with its own subcategories and intensity columns. The split is driven by the factor's context field, which is derived from the factor's extension.

How it classifies

Output file	Factors it includes
trade_employment.csv	context == economic/employment. Subcategories: People (Employment people:) and Hours (Employment hours:).
trade_resource.csv	context in {emission/water, natural_resource/water, natural_resource/land, natural_resource/energy} or fullname matches Crops / Primary Crops / Agriculture / Forestry / Fishery. Subcategories: Water_Consumption, Water-Withdrawal, Energy, Land_Crops, Land_Forest, Land_Other, Crops.
trade_material.csv	context == natural_resource/in_ground AND NOT in crops keywords. Subcategories: Metals, Minerals, Fossil, Other_Materials.

Column shape of each output

All three files share the same shape:

- **Core trade columns:** trade_id, year, region1, region2, industry1, industry2, amount.
- **Summary:** total_<category>_value, <category>_count, unique_<category>_factors.
- **Context pivot:** one column per context value found.
- **Subcategory columns:** as listed above.
- **Intensity:** <category>_intensity = total_value / amount.

Why three files instead of one

The split exists partly for file size (each file is manageable for Node.js and SQL clients) and partly because consuming dashboards tend to specialise — an employment dashboard does not need water columns, and a materials dashboard does not need employment. Like `trade_impact.csv`, these files are *computable from the base tables* and are therefore flagged as redundant in `PLAN.md`.

CHAPTER 8

main.py: Batch orchestration

`main.py` is the batch runner. It reads `config.yaml`, resolves the country list, and for each (country, flow_direction) tuple invokes the three processing scripts in order:

```
python trade.py          # produces trade.csv + trade_factor(_lg).csv
python trade_impact.py   # produces trade_impact.csv
python trade_resource.py # produces trade_resource.csv, _material.csv, _employment.csv
```

Three-tier timeouts

Tier	Limit	Behaviour
Per script	20 minutes (1200 s)	<code>subprocess.run(timeout=1200)</code> . Kills a hung <code>trade.py</code> or <code>trade_impact.py</code> .
Per country	60 minutes (3600 s)	Rolling budget across the three scripts for one country. Printed as a countdown.
Per batch	5 hours (18,000 s)	Hard ceiling for the whole run. The most restrictive timeout wins.

Resumability

After a country completes, `main.py` writes a `runnote.md` in that country's output folder. On the next run, if `runnote.md` exists, the country is **skipped**. This is how large multi-country runs survive crashes and partial failures — just restart and the batch resumes from where it stopped. During a run, the file is called `runnote-inprogress.md` and tracks intermediate state; on success it is renamed to `runnote.md`.

Observed timing

From the README's measured runs on 2019/US:

Flow	trade.py	trade_impact.py	trade_resource.py	Rows (trade / factor)
------	----------	-----------------	-------------------	-----------------------

exports	2m 14s	5.3s	9.0s	188,735 / 125,148
imports	2m 11s	3.5s	5.6s	126,166 / 19,425
domestic	2m 18s	1.7s	1.9s	21,518 / 11,832

CHAPTER 9

factor.csv and industry.csv

These two files live at the top of the `year/{year}/` tree, not inside a country folder — they are shared across all countries and flow directions. Both are generated once and reused.

factor.csv

Generated by `factors.py`. The script walks the six extensions in a fixed order (**air_emissions**, **employment**, **energy**, **land**, **material**, **water**) and assigns a 1-based `factor_id` to each stressor row in each extension's F matrix. The output has four columns:

- **factor_id** — stable integer PK. Order is deterministic given a fixed EXIOBASE release.
- **unit** — the unit string from EXIOBASE's extension metadata.
- **stressor** — the full stressor name (e.g. `C02 - combustion - air`).
- **extension** — which of the six extensions the row came from.

The total is **721 factors** for the 2019 release. The exact count can shift between EXIOBASE releases, which is why `factor_id` is *stable per release* but not *stable across releases*.

industry.csv

Generated by `create_sector_mapping.py`. The script reads the EXIOBASE sector list and converts each sector name to a 5-character **industry_id** via three strategies:

- **Strategy 1**: Strip common words (and, of, related, services, products, nec, other), remove punctuation, uppercase, take the first 5 characters.
- **Strategy 2**: If strategy 1 is too short, build an acronym from first letters of words.
- **Strategy 3**: If still too short, pad with zero-filled index.
- **Collision resolution**: if the resulting code is already used, replace the last character with a counter (then last two).

The output has three columns: **industry_id**, **name** (the full EXIOBASE sector name), and **category** (one of ~24 keyword-matched buckets: Agriculture, Forestry, Mining, Chemicals, Utilities, Trade, etc.).

Why 5 characters

The 5-character width is a deliberate design choice for consistency with other Model.Earth tables. The BEA README notes a planned 6-character **commodity** table for USEEIO / BEA / NAICS codes — those are structurally different taxonomies with their own stability concerns, so they get their own tables rather than being squeezed into the 5-character industry_id.

CHAPTER 10

The 120-vs-721 selection problem

This is the single most important design trade-off in the pipeline, and it drives a surprising amount of surface area — a config flag, two different output filenames, a smart file selector, and a warning in every single `trade_resource.py` run.

The numbers

Each trade transaction can potentially have a non-zero coefficient for all 721 environmental factors. For a US exports run with 188,735 trade transactions, that would produce $188,735 \times 721 = 136$ **million** rows in `trade_factor.csv`, before filtering. The resulting file is ~1.5 GB and routinely crashes Node.js consumers with *FATAL ERROR: v8::ToLocalChecked Empty MaybeLocal*.

The 120 selection

To keep the file manageable, the pipeline selects **120 factors per industry** by ranking all stressors whose absolute S-matrix coefficient meets `min_impact_threshold` (default 0.001) in descending order and keeping the first `partial_factor_limit` (default 120). $120 / 721 = 16.6\%$, so the output is about one-sixth the size of the full join.

Where each file is used

File	When generated
<code>trade_factor.csv</code>	Always for imports and exports. Also for domestic if <code>use_partial_factors_domestic</code> is true.
<code>trade_factor_lg.csv</code>	Only when <code>-lag</code> flag is passed, or when <code>use_partial_factors_domestic: false</code> .
<code>interstate_factor.csv</code>	US BEA state-to-state. Default is 50 selected factors (not 120; smaller because the row count is multiplied by state-pair combinatorics).
<code>interstate_factor_lg.csv</code>	When <code>use_partial_factors_interstate: false</code> .

The trade-off

Selecting the top 120 by coefficient magnitude optimises for **captured value** but can hide **specific stressors a user cares about**. If someone asks ‘what is the embodied methane in cement imports?’ and methane did not make the top-120 for that industry, the answer in `trade_factor.csv` will be 0 — silently. The `_lg` file is the escape hatch, but requires explicit opt-in and a willingness to deal with multi-gigabyte files.

CHAPTER 11

CSV output catalogue

A single python `main.py` run against `2019/US/exports` produces the following tree. Shared reference files sit at the year level; per-flow files sit under the `country/flow` folder.

```
year/2019/
  factor.csv                # 721 rows
  industry.csv              # ~200 rows
  US/
    exports/
      trade.csv              # 188,735 rows
      trade_factor.csv       # 125,148 rows (120 selected)
      trade_impact.csv       # 188,735 rows, 22 cols
      trade_resource.csv     # resources view
      trade_material.csv     # materials view
      trade_employment.csv   # employment view
      runnote.md
    imports/
      trade.csv              # 126,166 rows
      trade_factor.csv       # 19,425 rows
      ...
    domestic/
      trade.csv              # 21,518 rows
      trade_factor.csv (or trade_factor_lg.csv)
      ...
```

Column reference

File	Columns
factor.csv	factor_id, unit, stressor, extension
industry.csv	industry_id, name, category
trade.csv	trade_id, year, region1, region2, industry1, industry2, amount
trade_factor.csv	trade_id, factor_id, level
trade_impact.csv	trade_id + 7 trade columns + ~15 impact columns
trade_resource.csv	trade columns + total_resources_value, resources_count, unique_resources_factors, context pivot, subcategories, intensity
trade_material.csv	same shape as trade_resource.csv but materials-only
trade_employment.csv	same shape, employment-only (People + Hours subcategories)
interstate.csv	interstate_id, year, region1, region2, industry1, industry2, amount, commodity_code, industry_code, economic_multiplier
interstate_factor.csv	interstate_id, factor_id, level

CHAPTER 12

US BEA: Interstate state-to-state flows

`bea/main.py` is a separate but closely related pipeline that disaggregates US national trade flows into state-to-state flows, using BEA input-output data and employment multipliers. The output follows the same shape as `trade` and `trade_factor`, but at the state level: **`interstate.csv`** and **`interstate_factor.csv`**.

Prerequisites

`bea/main.py` is *not* standalone — it builds on top of the base pipeline. Before running, you need:

- `exiobase_data/IOT_{year}_pxp.zip` — the EXIOBASE download.
- The three US `trade.csv` files (domestic, imports, exports) — generated by `trade.py` or by passing `--force-regen`.
- A BEA API key in `webroot/.env` as `BEA_API_KEY=...` (register at apps.bea.gov/api/signup).

Five-phase pipeline

Phase	What it does
1. Base	Ensures the three Exiobase <code>trade.csv</code> files exist. If not, runs <code>trade.py</code> inline.
2. BEA API	<code>main_api_client.py</code> fetches <code>IntlServTrade</code> , <code>InputOutput</code> , and <code>GDPbyIndustry</code> data. Caches responses in <code>bea_cache/</code> for 24 hours.
3. State-level	<code>main_trade_analyzer.py</code> disaggregates national flows across states using BEA employment and output weights.
4. FEDEFL	<code>main_fedefl_integration.py</code> cross-walks EXIOBASE stressors to the Federal LCA Commons Elementary Flow List — write-only artefact for review.
5. S matrix join	Loads the EXIOBASE S matrix directly via <code>pymrio</code> to assign <code>factor_id</code> and compute per-flow levels for <code>interstate_factor.csv</code> .

The interstate_id key

Instead of a surrogate integer ID, `interstate_id` is a composite string:

```
interstate_id = {year}-US-{origin_state}-US-{destination_state}-{state_industry_code}
example: 2019-US-NY-US-CA-311
```

This makes the ID stable across runs and self-documenting at the cost of being long. It also means two tables — `interstate` and `interstate_factor` — share the composite key as the join column instead of needing a surrogate FK.

50 factors, not 120

Interstate flows have another combinatorial multiplier — 51 states × 51 states × industries × factors. At 120 factors per industry, the full table hit ~5 GB. The default is tightened to **50 selected factors** (`partial_factor_limit_interstate`: 5 wait, verify — actually the docs say 50 is the current operational default while `config.yaml` shows 5; the smaller value is for current test runs). The investigation note dated 2026-03-27 records 68.5M rows produced with 120 factors.

BEA economic_multiplier

`interstate.economic_multiplier` is the BEA Input-Output **Total industry output requirement** from TableID 61, keyed by BEA Summary `industry_code`. It is used to estimate total output impact as `interstate.amount × economic_multiplier`. When BEA has no matching row, the fallback is 1.0.

CHAPTER 13

India: State-level allocation

`india/main.py` is a parallel disaggregation pipeline that allocates Indian national trade and economic data across 36 states and union territories. Where the BEA pipeline uses BEA API data, the India pipeline uses raw data files stored under `exiobase/India_data/` — GSDP, GSVA, SUT/IOT, and TradeStat exports/imports.

Five allocation outputs

- **state_sector_output.csv** — State × sector output; GSDP scaled by GSVA sector shares.
- **state_product_export.csv** — National exports allocated to states, with HS codes mapped to EXIOBASE products.
- **state_product_import.csv** — National imports allocated by GSDP proportions.
- **india_states.csv** — State summary with Output, Employment, Population columns.
- **allocation_report.md** — Processing summary.

Shared taxonomy with US

The India pipeline deliberately uses the same 5-character **industry_id** values as the EXIOBASE core pipeline. The crosswalk file `india_us_exiobase_crosswalk.csv` maps Indian GSVA activity labels and HS product codes to EXIOBASE industry IDs. This means India and US outputs can be joined on `industry_id` for cross-country comparison without a separate harmonisation step.

Data categorisation

At startup, `india/main.py` scans `India_data/` and categorises files by keyword matches in the filename: anything containing `gsdp/sdp/gdp` is treated as GSDP data, anything with `gsva/nsva/value_added` as GSVA, and so on. State-specific Excel files are used as fallback when consolidated GSDP or GSVA files are not present.

CHAPTER 14

SQL ingestion (insert_sql.py)

insert_sql.py is a small, generic CSV-to-SQL loader that reads any of the generated CSVs and appends rows to a Postgres or MS SQL table, creating the table from the CSV schema on first run.

How it is invoked

```
python insert_sql.py \  
  --conn_id exiobase \  
  --connections_path ../Connections/db_connections.yaml \  
  --source_csv year/2019/US/exports/trade.csv \  
  --table_name trade \  
  --schema public \  
  --chunksize 5000
```

Connection resolution

Credentials come from a db_connections.yaml file (located one level up at Connections/db_connections.yaml). The YAML has one entry per connection ID with fields type, host, database, user, password, port. Supported types: **postgres** (via psycopg2) and **mssql** (via pyodbc with ODBC Driver 18).

The method='multi' detail

The call uses `df.to_sql(..., method='multi', chunksize=5000)`. This batches rows into multi-row INSERT statements rather than per-row inserts, which is roughly 10-20× faster for the Python side of the load. For truly large loads (interstate_factor at 68M rows), neither Python nor SQLAlchemy is the right path — you want `COPY FROM STDIN` via psycopg2 directly or `bcp` on MSSQL. The script does not do this currently.

Ingestion order

Foreign keys dictate the sequence. From PLAN.md:

- 1. factor.csv → factor table.

- **2.** industry.csv → industry table.
- **3.** For each (country, flow_type): trade.csv → trade; then trade_factor.csv → trade_factor.
- **4.** US only, domestic: interstate.csv → interstate; state_trade_flows.csv → interstate_factor.

CHAPTER 15

Running it: commands, timing, resumability

One-time setup

```
python -m venv ~/env
source ~/env/bin/activate      # macOS/Linux
~/env/Scripts/activate.bat    # Windows
pip install --prefer-binary -r requirements.txt
```

Standard run

```
# Edit config.yaml: set YEAR, TRADEFLOW, COUNTRY.list
cd exiobase/tradeflow
python main.py
```

The first run for a new year will spend ~10 minutes downloading the EXIOBASE zip (~535 MB) from Zenodo into `exiobase_data/`. That file is `.gitignored` and is reused across subsequent runs.

Partial runs

```
# Run just the extraction stage
python trade.py

# Run just the aggregation
python trade_impact.py

# Run just the resource split
python trade_resource.py

# Force the large 721-factor file for domestic
python trade.py -lag
```

US interstate

```
# Prerequisites: trade.csv files for US/{domestic,imports,exports}
python bea/main.py                                # uses BEA_API_KEY from webroot/.env
python bea/main.py --bea-key YOUR_KEY             # explicit key
python bea/main.py --force-regen                  # regenerate base trade.csv first
```

India state-level

```
python india/main.py                                # uses config.yaml year
python india/main.py --year 2019
python india/main.py --data-dir ../India_data
```

Resume behaviour

If `main.py` is killed mid-run or hits a timeout, it leaves `runnote-inprogress.md` files in incomplete country folders. On the next run those countries are retried; countries with a final `runnote.md` are skipped. To **force** a re-run for a specific country, delete its `runnote.md`.

CHAPTER 16

Troubleshooting and gotchas

Memory errors in Node.js consumers

FATAL ERROR: v8::ToLocalChecked Empty MaybeLocal after about 10 minutes is the signature of a Node consumer trying to parse `trade_factor_lg.csv` (~1.5 GB). The fix is to use the default 120-factor file, not the `-lg` version, for the flow being consumed. If domestic analysis *needs* the full 721 factors, process it in chunks or move to SQL.

‘trade_factor.csv not found’

`trade_impact.py` and `trade_resource.py` both expect `trade_factor.csv` as input. If it is missing, `trade.py` never completed. Check the runnote for the stage that failed.

Empty domestic flows

If domestic processing reports zero flows, the country code probably does not match an EXIOBASE region. Verify the country code is one of the ~44 EXIOBASE codes (two-letter codes such as US, CN, DE, plus five rest-of-world codes). `debug_regions.py` can be used to print the full list.

Employment numbers off by 1,000×

If employment totals look wrong by three orders of magnitude, the `trade_impact.py` fix documented in `EMPLOYMENT_FIX_SUMMARY.md` was not applied. Employment people factors use **1000 p** units and need a $\times 1000$ conversion to actual people; the hours factors use **M.hr** and stay in millions.

Zenodo URL changes

EXIOBASE is hosted on Zenodo under DOI 10.5281/zenodo.3583070. Zenodo has, in the past, changed its download URL format from `/records/ID/files/NAME.zip` to `/api/records/ID/files/NAME.zip/content`. The `_download_direct()` method in `trade.py` bypasses pymrio's built-in downloader for this reason — if downloads fail with 404, the Zenodo URL format has probably changed again.

factor_id does not match stressor

Short stressor names like C02 are ambiguous — there are multiple CO2 factor rows in different contexts (combustion, non-combustion, biogenic). `trade.py` builds a robust mapping that handles common formatting variations (PM2.5 vs PM2_5, dots vs underscores) but will log *'unmapped stressors'* warnings if a factor name cannot be matched. Those rows are dropped from `trade_factor.csv`.

CHAPTER 17

The schema being proposed for SQL

PLAN.md describes the target SQL schema. It is not yet implemented in a database — the CSVs are the current source of truth — but the shape is settled enough that the ingestion endpoints have been scoped. Three tables hold the base data; three more hold the interstate extension.

Base tables

```
industry (industry_id PK VARCHAR(10), name TEXT, category VARCHAR(100))
```

```
factor (factor_id PK INTEGER, unit VARCHAR(50), stressor TEXT, extension VARCHAR(100))
```

```
trade (  
  id          BIGSERIAL PRIMARY KEY,  
  trade_id    INTEGER NOT NULL,  
  year        SMALLINT NOT NULL,  
  region1     VARCHAR(10) NOT NULL,  
  region2     VARCHAR(10) NOT NULL,  
  industry1   VARCHAR(10) REFERENCES industry(industry_id),  
  industry2   VARCHAR(10) REFERENCES industry(industry_id),  
  amount      NUMERIC(18,4),  
  flow_type   VARCHAR(10) NOT NULL,  
  country     VARCHAR(10) NOT NULL,  
  UNIQUE (trade_id, year, country, flow_type)  
)
```

Trade_factor

```
trade_factor (
  id          BIGSERIAL PRIMARY KEY,
  trade_id    INTEGER NOT NULL,
  year        SMALLINT NOT NULL,
  country     VARCHAR(10) NOT NULL,
  flow_type   VARCHAR(10) NOT NULL,
  factor_id   INTEGER NOT NULL REFERENCES factor(factor_id),
  coefficient  NUMERIC(20,10),
  level       NUMERIC(20,6)
)

-- Join to trade:
-- trade_factor.trade_id = trade.trade_id
--   AND trade_factor.year = trade.year
--   AND trade_factor.country = trade.country
--   AND trade_factor.flow_type = trade.flow_type
```

Notes on the proposed shape

- **Composite join:** because `trade_id` from CSV is per-run and per-flow, the join to `trade` must include `(year, country, flow_type)` to disambiguate.
- **Redundant tables omitted:** `trade_impact`, `trade_resource`, `trade_material`, `trade_employment` are not planned as physical tables — they are computed via JOIN at query time.
- **Indexes** on `trade_id`, `year`, `country`, `flow_type` for both `trade` and `trade_factor`; separate indexes on `region1`, `region2`, `factor_id`.
- **Rust API:** three endpoints — `/api/db/init-industry-tables`, `/api/db/insert-trade-data`, `/api/db/industry-schema` — are planned to drive the SQL side.

Interstate tables

```
interstate (  
  id          BIGSERIAL PRIMARY KEY,  
  trade_id    INTEGER NOT NULL,  
  year        SMALLINT NOT NULL,  
  region1     VARCHAR(10) NOT NULL, -- US-AK  
  region2     VARCHAR(10) NOT NULL, -- US-GA  
  industry1   VARCHAR(10) REFERENCES industry(industry_id),  
  industry2   VARCHAR(10) REFERENCES industry(industry_id),  
  amount      NUMERIC(18,4),  
  commodity_code VARCHAR(30),  
  industry_code VARCHAR(30),  
  economic_multiplier NUMERIC(10,6)  
)  
  
interstate_factor (  
  id          BIGSERIAL PRIMARY KEY,  
  interstate_id VARCHAR(80) NOT NULL,  
  factor_id   INTEGER REFERENCES factor(factor_id),  
  level       NUMERIC(20,6)  
)
```

CHAPTER A

Repository map

The exiobase/tradeflow/ directory, annotated:

```

tradeflow/
  AGENTS.md                # Full developer notes for Claude Code
  PLAN.md                  # Target SQL schema and ingestion plan
  README.md                # Public-facing overview
  REMOVED.md               # Scripts removed from the pipeline
  EMPLOYMENT_FIX_*.md      # Record of the 1000-persons unit bug fix
  requirements.txt          # Python dependencies
  config.yaml              # Central configuration
  config-bkup.yaml         # Last-known-good config backup

# --- CORE PIPELINE (invoked by main.py) ---
main.py                    # Batch orchestrator with 3-tier timeouts
trade.py                   # Z + S matrix extraction, factor selection
trade_impact.py            # Per-trade impact aggregation (22 columns)
trade_resource.py          # Split into employment / resource / material
config_loader.py           # YAML + env-var resolution

# --- SUPPORT ---
factors.py                 # Generate factor.csv from F matrix
create_sector_mapping.py   # Generate industry.csv from Z sectors
create_full_trade_factor.py # Historical 721-factor generator (legacy)
insert_sql.py              # Generic CSV & Postgres/MSSQL loader

# --- UTILITIES ---
update_current_country.py  # Set config.yaml COUNTRY.current
process_countries_sequentially.py # Alternative orchestrator
process_remaining.py       # Resume helper
run_single_country.py      # Convenience for one country
run_multiple_countries.py  # Convenience for several
run_batch_imports.py       # Imports-only batch
run_domestic_batch.py      # Domestic-only batch
debug_regions.py           # Print EXIOBASE region codes
examine_factors.py         # Inspect factor.csv contents
examine_sectors.py         # Inspect sector codes
test_config.py             # Config loader unit tests
test_trade_factors.py       # Factor generation smoke tests
trade_competitiveness.py   # Export RCA analysis (standalone)

# --- US BEA EXTENSION ---
bea/
  main.py                  # Interstate pipeline endpoint
  main_api_client.py       # BEA API with 24-hour cache
  main_trade_analyzer.py   # State disaggregation + impacts
  main_fedefl_integration.py # FEDEFL flow metadata

```

```
    README.md          # BEA-specific docs
    PLAN.md            # Outstanding BEA questions

# --- INDIA EXTENSION ---
india/
  main.py              # India state allocation
  README.md            # India pipeline docs

# --- WEB ---
index.html             # Browser landing page
```

CHAPTER B

Glossary

Term	Definition
amount	Monetary value of a trade transaction in million euros (M EUR). Column in <code>trade.csv</code> and <code>interstate.csv</code> .
category	One of ~24 industry buckets (Agriculture, Mining, Chemicals, Utilities, ...) derived by keyword matching the EXIOBASE sector name. Column in <code>industry.csv</code> .
coefficient	Per-M-EUR intensity from the EXIOBASE S matrix. Not stored in output CSVs — derivable as <code>level / amount</code> .
context	A derived category string like <code>emission/air</code> or <code>natural_resource/water</code> , used in <code>trade_resource.py</code> to split factors.
domestic	Flow direction: intra-country trade (<code>region1 == region2 == country</code>).
exports	Flow direction: outbound from the configured country (<code>region1 == country, region2 != country</code>).
extension	One of six EXIOBASE environmental extensions: <code>air_emissions</code> , <code>employment</code> , <code>energy</code> , <code>land</code> , <code>material</code> , <code>water</code> .
factor	A single environmental stressor row. 721 total across six extensions.
factor_id	1-based integer PK for a factor. Order determined by extension order and row order within each extension's F matrix.
FEDEFL	Federal LCA Commons Elementary Flow List — EPA's canonical environmental flow registry. Mapped to EXIOBASE factors in <code>flow.csv</code> (review-only artefact).
flow (deprecated term)	What EXIOBASE calls a Z-matrix cell. TradeFlow renames this to trade to avoid confusion with Sankey arcs and FEDEFL flows.
imports	Flow direction: inbound to the configured country (<code>region1 != country, region2 == country</code>).

industry	A product-by-product EXIOBASE sector. ~200 total.
industry_id	5-character mnemonic PK for an industry (e.g. WHEAT, CEREAL, PADDY). Generated by name-truncation with collision counter.
interstate	A state-to-state analogue of trade, produced by <code>bea/main.py</code> .
interstate_id	Composite string ID: {year}-US-{origin}-US-{dest}-{code}. Used as the join column between interstate and interstate_factor.
level	Physical impact quantity in <code>trade_factor.csv</code> . Equal to <code>amount × coefficient</code> . Unit depends on the factor's extension.
MRIO	Multi-regional input-output. The economic modelling framework EXIOBASE implements.
partial_factor_limit	Config knob setting how many factors per industry survive the selection filter. Defaults: 120 for international, 50 for interstate.
pymrio	The Python library used to parse EXIOBASE zip files into pandas-backed objects with <code>.Z</code> , <code>.Y</code> , <code>.A</code> , and extension attributes.
runnote.md	Per-country marker file. If present in a country's output folder, <code>main.py</code> skips that country on the next run.
S matrix	Per-M-EUR intensity matrix for an environmental extension. $S = F / x$ where x is total output per sector.
stressor	Full factor name as it appears in EXIOBASE (e.g. C02 - combustion - air). Column in <code>factor.csv</code> .
trade	A row in <code>trade.csv</code> : one bilateral industry-pair flow in a given year and direction.
trade_id	1-based integer ID for a trade row. Per-run, per-country, per-direction — <i>not</i> globally stable.
Y matrix	Final demand matrix. Not currently written to output CSVs by this pipeline (<code>industryflow_finaldemand.py</code> was removed per REMOVED.md).
Z matrix	Inter-industry transaction matrix in M EUR. Source of all <code>trade.csv</code> rows.

— *End of Field Guide* —

Source files are in `exiobase/tradeflow/`. When documentation and code disagree, trust the code.